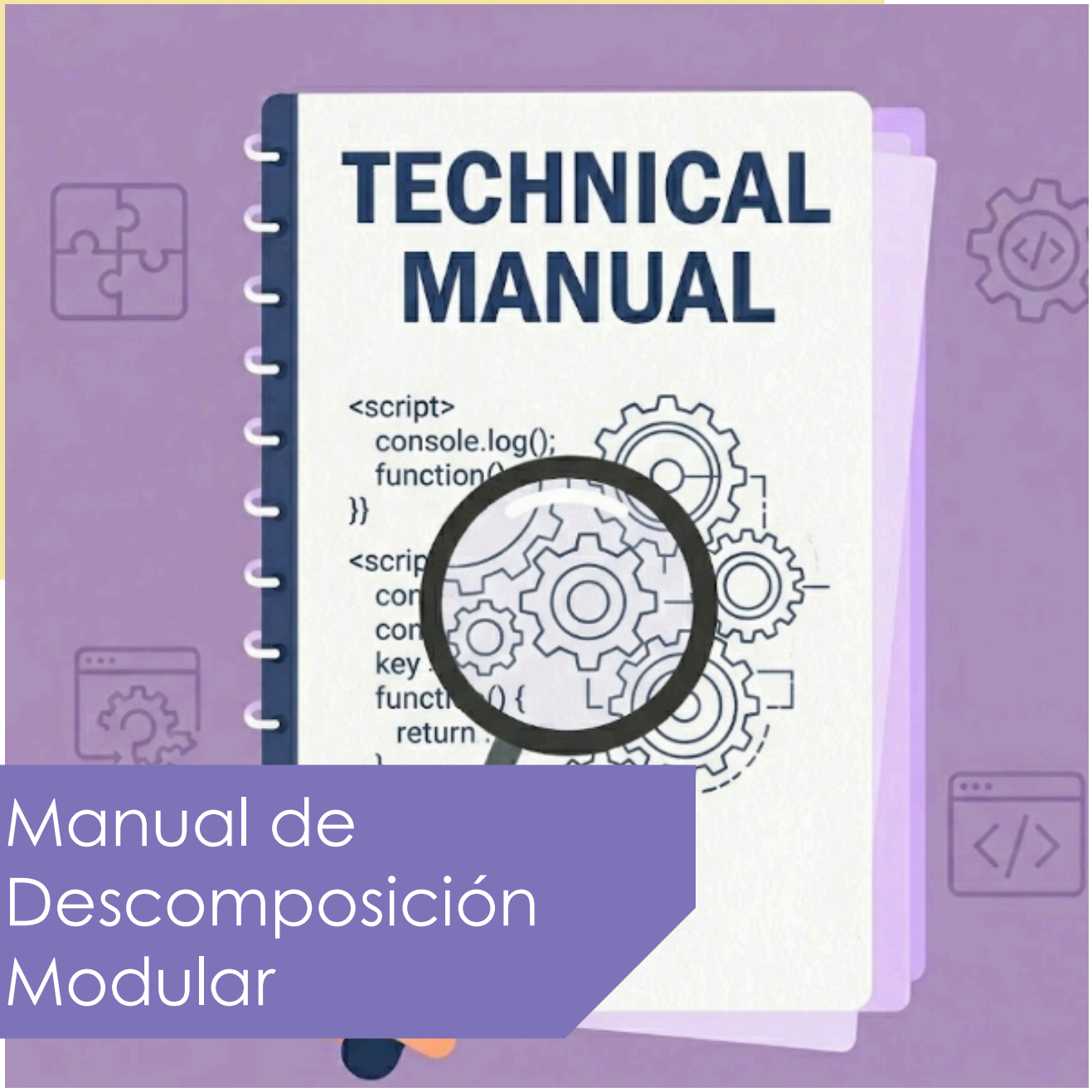


Manual de Descomposición Modular

Lógica 101

SebastiánCastilloMartinez
Gustavo Adrián Joya Rodriguéz
Jesus Daniel Marín Terrazas

TABLA DE CONTENIDO

Introducción_____ 3
Arquitectura General del sistema_____ 4
Diagrama de Módulos _____ 6
Explicación de Clases y Métodos _____ 8
Algoritmo del Renglón Crítico _____ 10
Algoritmo de verificación de tautología_____ 11
Manejo de Errores _____ 12
Estructuras de Datos Utilizadas _____ 13
Conclusión Técnica_____ 14

INTRODUCCIÓN

Este documento describe la descomposición modular del sistema, enfocándose en su arquitectura interna, organización de módulos y estructura funcional. Se buscara detallar cómo interactúan las clases, métodos y componentes lógicos que permiten la ejecución del programa.

El sistema fue dividiendo las funciones del programa, separando la parte visual (interfaz gráfica) de la parte que realiza los cálculos lógicos. Mientras la ventana solo muestra información y recibe datos, mientras que el motor lógico se encarga únicamente de procesar y validar los argumentos. Esta organización facilita entender el programa, corregir errores y hacer futuras mejoras.

Asimismo, se explican los algoritmos implementados para el Método del Renglón Crítico y la Verificación por Tautología, así como el manejo de errores y las estructuras de datos utilizadas. Con esto proporcionando una visión completa del diseño y funcionamiento interno del proyecto.

Arquitectura general del sistema

El sistema está organizado en una estructura modular, es decir esta dividido en módulos distintos donde cada uno cumple una función específica: Esta división se refleja en el código, donde la clase **SuiteLogica,** esta encargada de la interfaz gráfica, de las funciones externas que realizan el procesamiento lógico. Así, se evita mezclar la parte visual con los cálculos.

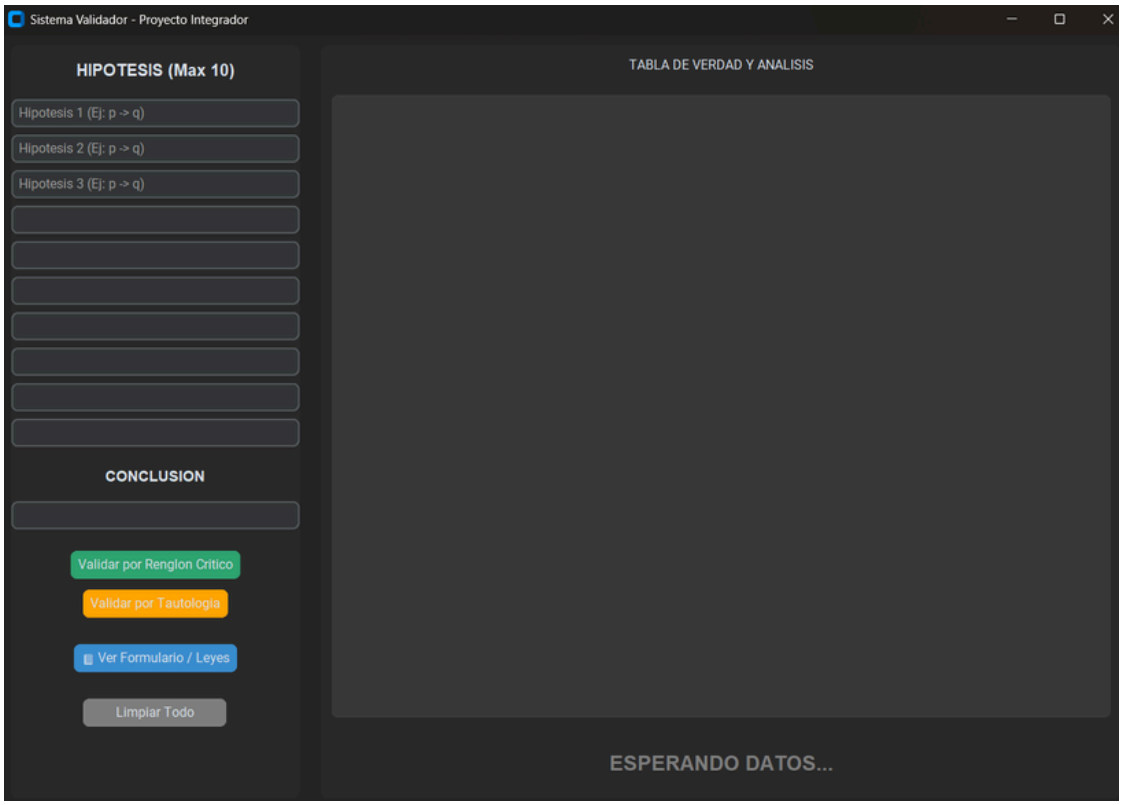


Figura 1. Interfaz principal del Sistema Validador.

La **Interfaz Grafica (Frontend)** usando la clase **SuiteLogica**, desarrollada con la librería **customtkinter**, construye la ventana principal del sistema y gestiona la interacción con el usuario. Dentro de esta clase se capturan las hipótesis y la conclusión, se controlan los botones de validación y se muestran los resultados obtenidos. Su función principal es coordinar la entrada y salida de datos, sin realizar directamente los cálculos lógicos.

Arquitectura general del sistema

El procesamiento lógico se realiza en el módulo denominado **Motor Lógico (Backend)**, el cual se encarga de interpretar y evaluar expresiones proposicionales, generar las combinaciones de verdad necesarias y verificar la validez de los argumentos ingresados. Este módulo opera de manera independiente de la interfaz gráfica, utilizando las librerías **sympy** e **itertools** para asegurar un análisis lógico formal y confiable.

El flujo general del sistema inicia cuando el usuario ingresa los datos y selecciona un método de validación. La clase **SuiteLogica** obtiene la información mediante el método **obtener_datos_seguro()**, genera las combinaciones con **generar_tabla_verdad()** y llama al algoritmo correspondiente. Finalmente, los resultados son enviados nuevamente a la interfaz para mostrarse en pantalla junto con el veredicto final del argumento

Diagrama de Módulos

El sistema está dividido en dos módulos principales: Motor Lógico (Backend) e Interfaz Gráfica (Frontend), cada uno tiene sus responsabilidades definidas y se comunican mediante llamadas a funciones específicas.

Modulo 1: Backend

Este módulo contiene todas las funciones encargadas del procesamiento matemático y la validación de argumentos lógicos. Entre sus funciones se incluye:

- **traducir_a_sympy()** = Traduce expresiones de notación humana a sintaxis de SymPy.
- **validar_y_extraer_vars()** = Extrae las variables de las fórmulas y verifica la sintaxis.
- **generar_tabla_verdad() =** Crea todas las combinaciones posibles de Verdadero/Falso para las variables.
- **evaluar_logica_renglon_critico()** = Aplica el método del renglón crítico para validar argumentos.
- **evaluar_logica_tautologia()** = Aplica el método de la tautología para validar argumentos.

Estas funciones utilizan la librería **sympy** para interpretar y evaluar expresiones proposicionales, y **itertools** para generar las combinaciones necesarias en la tabla de verdad.

Modulo 2: Fronted

Implementado mediante **SuiteLogica** con **customtkinter**, este módulo se encarga de gestionar la interacción con el usuario, mostrar resultados y coordinar la entrada y salida de datos. Entre sus métodos principales se incluyen:

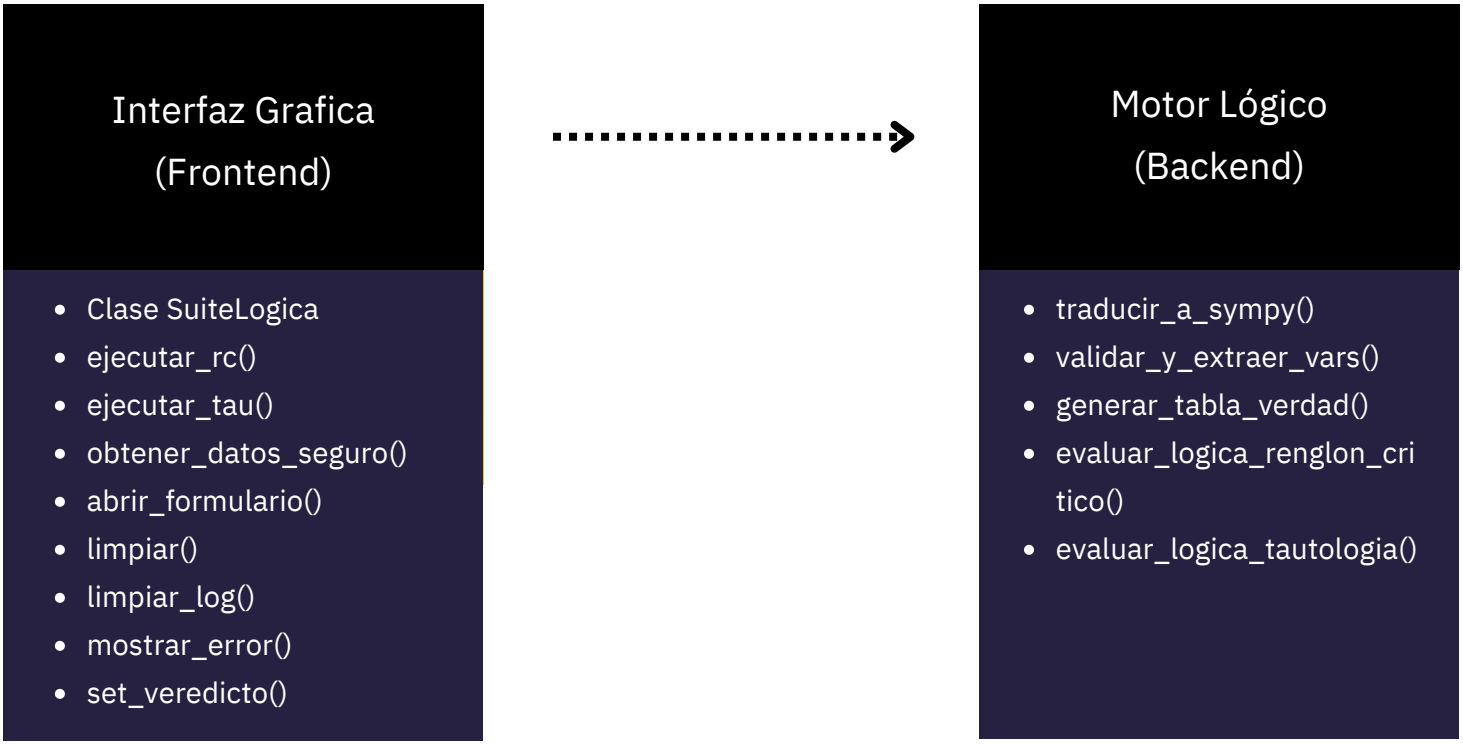
- **ejecutar_rc()** = Valida argumentos con el método del renglón crítico y muestra resultados.
- **ejecutar_tau()** = Valida argumentos con el método de la tautología y muestra resultados.
- **obtener_datos_seguro()** = Recolecta y valida los datos ingresados por el usuario.
- **abrir_formulario()** = Abre una ventana con el formulario de leyes y reglas de lógica.
- **limpiar()** = Borra todas las entradas y resultados.
- **limpiar_log()** = Borra únicamente los resultados.
- **mostrar_error()** = Muestra mensajes de error en pantalla.
- **set_veredicto()** = Actualiza el veredicto final y su color.

Diagrama de Módulos

Comunicación entre módulos

El flujo de información sigue estos pasos:

1. La interfaz obtiene los datos del usuario y los valida con **obtener_datos_seguro()**.
2. Se generan las combinaciones de la tabla de verdad mediante **generar_tabla_verdad()**.
3. Se llama al algoritmo correspondiente del Motor Lógico (**evaluar_logica_renglon_critico** o **evaluar_logica_tautologia**).
4. Los resultados se envían de vuelta a la interfaz para su visualización, incluyendo la tabla de verdad y el veredicto final.



Explicación de cada clase/método

Modulo 1: Backend

traducir_a_sympy(expresion)

- **Propósito**: Traduce expresiones de notación humana a sintaxis de SymPy.
- **Entrada**: Cadena de texto con símbolos como ->, v, ^, <->.
- **Salida**: Cadena de texto convertida a operadores de Python.
- **Detalle**: Realiza un preprocesamiento del bicondicional <-> transformándolo en una conjunción de implicaciones para que la librería pueda interpretarlo correctamente.

validar_y_extraer_vars(texto_caja)

- **Propósito**: Extrae las variables de las fórmulas y verifica la sintaxis.
- **Entrada**: Texto ingresado en una de las cajas de hipótesis o conclusión.
- **Salida**: Un conjunto (set) con las letras proposicionales encontradas.
- **Detalle**: Actúa como un escudo de seguridad; si la fórmula tiene errores de escritura, lanza un ValueError para evitar que el programa falle.

generar_tabla_verdad(variables)

- **Propósito**: Crea todas las combinaciones posibles de Verdadero/Falso para las variables.
- **Entrada**: Lista de variables únicas detectadas en el argumento.
- **Salida**: Lista de tuplas con todas las combinaciones booleanas.
- **Detalle**: Utiliza **itertools.product** para generar la tabla de verdad, generando 2^n posibilidades según el número de variables.

evaluar_logica_renglon_critico(hipotesis, conclusion, valores_diccionario)

- **Propósito**: Aplica el método del renglón crítico para validar argumentos.
- **Entrada**: Lista de hipótesis, la conclusión y un diccionario de valores (V/F).
- **Salida**: Resultados individuales de cada premisa, el valor de la conclusión y la bandera de "renglón crítico".
- **Detalle**: Identifica si una fila tiene todas sus hipótesis verdaderas y comprueba si la conclusión también lo es.

evaluar_logica_tautologia(hipotesis, conclusion, variables, tabla)

- **Propósito**: Aplica el método de la tautología para validar argumentos.
- **Entrada**: Hipótesis, conclusión, variables y la tabla de verdad completa.
- **Salida**: La fórmula generada, el log de texto y el veredicto booleano.
- **Detalle**: Construye la fórmula (H1^H2...^Hn) -> C y verifica que sea una tautología absoluta.

Explicación de cada clase/método

Modulo 2: Frontend

__init__()

- **Propósito:** Construye la ventana principal y organiza los paneles de entrada y salida.
- **Detalle:** Configura el entorno visual "Dark/Green", genera dinámicamente las 10 cajas de texto para hipótesis y prepara el área de resultados.

ejecutar_rc() / ejecutar_tau()

- **Propósito:** Valida argumentos con sus respectivos métodos y muestra resultados**.**
- **Detalle:** Estas funciones son el enlace entre el usuario y el motor lógico. Invocan la validación, procesan la respuesta y dibujan la tabla de verdad en el panel derecho.

obtener_datos_seguro()

- **Propósito:** Recolecta y valida los datos ingresados por el usuario.
- **Entrada:** Datos crudos de los campos **CTkEntry.**
- **Salida:** Tupla con hipótesis limpias, conclusión y variables ordenadas.
- **Detalle:** Filtra campos vacíos y asegura que el argumento tenga al menos una premisa y una conclusión antes de procesar.

abrir_formulario()

- **Propósito:** Abre una ventana con el formulario de leyes y reglas de lógica.
- **Detalle:** Despliega una ventana emergente de apoyo con el contenido teórico necesario para que el usuario pueda verificar sus ejercicios.

limpiar() / limpiar_log()

- **Propósito:** Borra las entradas del usuario o únicamente los resultados**.**
- **Detalle:** Permite resetear la interfaz para comenzar un nuevo análisis sin necesidad de reiniciar la aplicación.

mostrar_error() / set_veredicto()

- **Propósito:** Gestión de mensajes y estados finales**.**
- **Detalle: mostrar_error avisa** sobre fallas de sintaxis, mientras que set_veredicto cambia el color de la interfaz (Verde/Rojo) según la validez del argumento.

Algoritmo del renglón crítico

El algoritmo se implementa en la función **evaluar_logica_renglon_critico()**. Su objetivo es determinar si un argumento es válido analizando únicamente los renglones de la tabla de verdad donde todas las hipótesis son verdaderas.

Primero, el sistema traduce las hipótesis y la conclusión a expresiones simbólicas utilizando **traducir_a_sympy()**. Después, se obtienen las variables proposicionales mediante **validar_y_extraer_vars()** y se generan todas las combinaciones posibles de valores de verdad con **generar_tabla_verdad()**.

Una vez generada la tabla de verdad, el algoritmo recorre cada combinación (renglón). Para cada renglón, evalúa las hipótesis y verifica si todas resultan verdaderas. Si esto ocurre, se considera un renglón crítico. En ese caso, se evalúa la conclusión bajo la misma combinación de valores.

Si existe al menos un renglón crítico donde las hipótesis sean verdaderas y la conclusión sea falsa, el argumento se considera inválido. Si en todos los renglones críticos la conclusión también es verdadera, el argumento se declara válido.

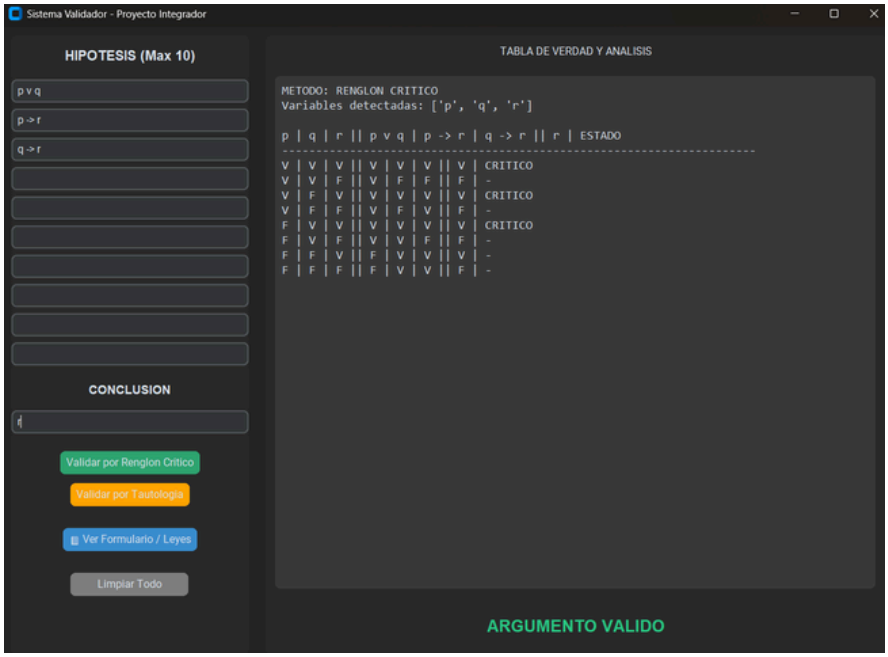


Figura 2. Ejecución del algoritmo de Renglón Crítico mostrando la tabla de verdad generada y el veredicto final del argumento.

Este método se basa directamente en la definición formal de validez lógica, un argumento es válido si no existe ninguna situación en la que las premisas sean verdaderas y la conclusión falsa.

Algoritmo de verificación de tautología

El algoritmo se implementa en la función **evaluar_logica_tautologia()**. Su objetivo es determinar si el argumento completo es una tautología, es decir, si la proposición resultante es verdadera en todas las combinaciones posibles de valores de verdad.

Primero, las hipótesis y la conclusión son traducidas a expresiones simbólicas mediante la función **traducir_a_sympy()**. Posteriormente, se obtienen las variables proposicionales con **validar_y_extraer_vars()** y se generan todas las combinaciones posibles utilizando **generar_tabla_verdad()**.

El sistema construye una expresión lógica que representa el argumento completo en forma de implicación, donde la conjunción de todas las hipótesis implica la conclusión. Esta expresión es evaluada en cada renglón de la tabla de verdad

Si en todas las combinaciones posibles la expresión resulta verdadera, el argumento se considera una tautología y, por lo tanto, válido. Si existe al menos una combinación donde la expresión sea falsa, entonces no es una tautología y el argumento se declara inválido.

Este método verifica directamente la definición formal de tautología, una proposición es tautológica cuando su valor de verdad es verdadero bajo cualquier asignación de valores a sus variables.

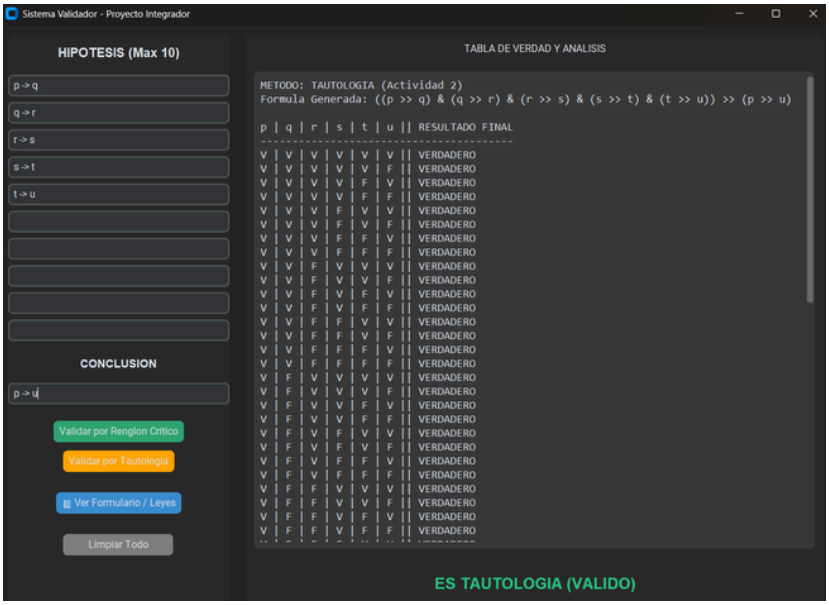


Figura 3. Ejecución del algoritmo de verificación de tautología mostrando la implicación global evaluada en todas las combinaciones posibles.

Manejo de errores

El sistema implementa un manejo de errores basado en la validación de entradas y control de excepciones durante el procesamiento lógico. Se evalúa las expresiones proposicionales ingresadas por el usuario, verifica tanto la integridad de los datos como la corrección gramatica antes de ejecutar los algoritmos de análisis.

En la capa de interfaz, el método **obtener_datos_seguro()** valida que los campos correspondientes a hipótesis y conclusión no se encuentren vacíos y que la estructura mínima del argumento esté presente. Si la validación falla, el flujo de ejecución se interrumpe, evitando evaluaciones innecesarias o errores posteriores.

En la capa de procesamiento, la función **traducir_a_sympy()** puede generar excepciones cuando la expresión lógica presenta errores de sintaxis, como conectores mal escritos, paréntesis desbalanceados o símbolos no reconocidos. Cuando esto ocurre, el sistema captura la excepción mediante estructuras **try/except**, muestra un mensaje indicando que la fórmula ingresada es incorrecta, evitando que el programa se cierre de manera inesperada y manteniendo el control del flujo de ejecución.

Este enfoque permite mantener un flujo de ejecución controlado, evitando que los errores en la entrada del usuario afecten otras partes del sistema. Garantizando la estabilidad general del programa, ya que solo procesa expresiones lógicas correctamente formadas.

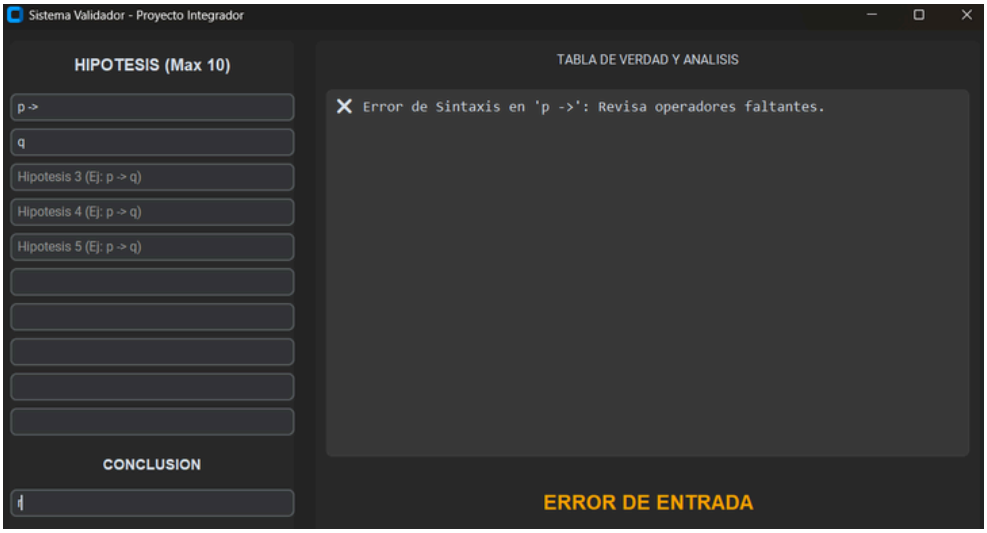


Figura 4. Detección y manejo de error de sintaxis durante la interpretación de la expresión lógica.

Estructuras de datos utilizadas

El sistema utiliza estructuras de datos básicas de Python para almacenar, organizar y procesar la información lógica ingresada por el usuario. Estas estructuras permiten manejar tanto expresiones proposicionales como las combinaciones de valores de verdad necesarias para la evaluación.

Las listas (list) se utilizan para almacenar las hipótesis ingresadas por el usuario y para gestionar conjuntos de expresiones lógicas durante el procesamiento. También para recorrer de manera ordenada los elementos durante la evaluación de cada renglón de la tabla de verdad.

Las tuplas (tuple) se generan mediante la función **product()** de la librería **itertools**, dentro de **generar_tabla_verdad()**. Estas tuplas representan cada combinación posible de valores booleanos (True/False) para las variables proposicionales, funcionando como los renglones de la tabla de verdad.

Los diccionarios (dict) se utilizan para asociar cada variable proposicional con un valor de verdad específico en cada combinación. Esto para evaluar las expresiones simbólicas sustituyendo cada variable por su valor correspondiente en un renglón determinado.

Además, el sistema emplea objetos simbólicos de **sympy**, que representan las expresiones lógicas. Estas estructuras permiten realizar evaluaciones booleanas, construir implicaciones y verificar condiciones de validez o tautología de manera formal.

El uso combinado de listas, tuplas, diccionarios y objetos simbólicos permite que el sistema genere la tabla de verdad, evalúe cada combinación y determine correctamente la validez lógica del argumento ingresado.

Conclusión tecnica

El sistema fue desarrollado bajo una estructura modular que separa claramente la interfaz gráfica del motor lógico, permitiendo un diseño organizado y mantenible. La clase **SuiteLogica** gestiona la interacción con el usuario, mientras que las funciones externas realizan el procesamiento simbólico y la evaluación formal de los argumentos.

La implementación de los algoritmos de Renglón Crítico y Verificación de Tautología, junto con el control de errores y el uso adecuado de estructuras de datos, garantiza un funcionamiento estable y coherente con los fundamentos de la lógica proposicional.

En conjunto, el sistema demuestra una correcta aplicación de principios de modularidad, separación de responsabilidades y evaluación lógica formal dentro de un entorno computacional.